Standard Operating Procedure

Running and post-processing multihit DEFORM simulations with run_deform_hits_config.py

1. Purpose

This SOP explains how to use the config-driven DEFORM multihit runner, what each referenced code file contributes, what outputs are expected, and how to post-process one completed run into microstructure, load, coordinate, damage, and summary-metric files.

- Primary run script: run_deform_hits_config.py
- Post-processing script: run_deform_post_processing.py
- Run environment: Spyder or command line on the lab computer with DEFORM API available
- Normal run unit: one configured multihit sequence, with hit 0 used for setup and hits 1..N used for forming

2. Code Files and Responsibilities

File	Role in the Workflow
run_deform_hits_config.py	Loads the JSON config/action CSV, selects the starting key, positions dies, writes and solves each hit DB, exports per-hit keys, coordinate logs, and load-stroke CSVs.
die_positioning.py	Seats top and bottom dies against the workpiece, backs them out by the configured height, and saves workpiece/die min, max, and center coordinates.
grain_initialization.py	Builds the DEFORM key string used to initialize JMAK grain size from supplier/target mean grain size before hit 1.
microstructure_post_processing.py	Parses DEFORM GRAIN blocks from KEY files into element-level CSV files and summarizes grain metrics.
metrics.py	Summarizes microstructure, load-stroke, coordinate, and damage CSVs into numeric metrics.
run_deform_post_processing.py	New single-run post-processor. Reads the completed run folder, parses all available outputs, combines logs, extracts damage when possible, and writes run summaries.

3. Required Inputs

Input	Where It Is Set	Notes
JSON config	--config argument or CONFIG_PATH in Spyder	Preferred for traceability. Controls paths, material, run name, number of hits, motion mode, output options, grain size, and column names.
Built-in DEFAULT_CONFIG	run_deform_hits_config.py	Allowed when no JSON config is passed. The post-processor can read this default directly from the runner source file.
Action CSV	action_path + action_file	Contains one row per planned hit/position with die, offset, rotation, forging temperature, speed, and depth.
Starting KEY file	init_key or key_path + key_files[die]	When use_jmak is true, the runner selects the _jmak.key version of the configured key template.
DEFORM API modules	Lab computer Python path	Required to run simulations and to extract DB-based damage. File-only post-processing can still parse existing CSV/KEY outputs.

Minimum config fields to check before a run:

```
{
  "key_path": "C:\\path\\to\\key_files",
  "action_path": "C:\\path\\to\\action_files",
  "action_file": "my_action_file.csv",
  "data_path": "C:\\path\\to\\runs",
  "material": "316L",
  "run_number": 1,
  "num_hits": 5,
  "height": 0.025,
  "use_jmak": true,
  "mean_grain_size_um": 26.3,
```

```
  "post_process_after_run": true
}
```

4. Before Running

- Confirm the action CSV has the required columns configured under config['columns'].

- Confirm the die column is present or set config['die'] explicitly.

- Confirm the selected key file exists. For JMAK runs, confirm the _jmak.key version exists.

- Use an action filename containing bdm when both dies should move symmetrically. The runner uses that naming rule when apply_action_file_rules is true.

- Keep position_dies_before_first_hit true unless you are deliberately testing a pre-positioned starting key.

5. Running the Multihit Simulation

Spyder procedure:

```
from run_deform_hits_config import main

CONFIG_PATH = r"C:\\path\\to\\run_config.json"
main(CONFIG_PATH)
```

Command line procedure:

```
python run_deform_hits_config.py --config C:\\path\\to\\run_config.json
```

The runner creates one run folder under data_path/run_name. Inside that folder, each hit has its own subfolder named run_name-hit<N>.

If using the runner's built-in DEFAULT_CONFIG:

```
from run_deform_hits_config import main

main()
```

What hit 0 does:

- Reads the base key file.

- Applies supplier grain-size initialization when use_jmak is true.

- Seats the dies against the workpiece and backs them out by the configured height.

- Writes the initial DB and initial key without solving a forming stroke.

What hits 1..N do:

- Read depth, speed, temperature, and position information from the action CSV.

- Build the down/up stroke timing plan.

- Apply symmetric die motion for bdm action files or single-top-die motion otherwise.

- Write and submit the DEFORM DB.

- Export final/post keys, coordinate logs, and load-stroke CSVs when enabled.

- Reposition the workpiece and reseat dies for the next hit.

6. Expected Runner Outputs

Output	Where	Purpose
af_hit<N>.DB	<run_name>-hit<N>/	DEFORM database for the hit.
af-hit-0_initial.KEY	<run_name>-hit0/	Initialized pre-solve key. This is the preferred source for initial microstructure.

Output	Where	Purpose
af-hit<N>_final.KEY	<run_name>-hit<N>/	Solved final state for that hit.
af-hit<N>_post.KEY	<run_name>-hit<N>/	Post-repositioned state for the next hit. For the last hit, post and final are intentionally identical.
object_coordinates_hit<N>_<stage>.csv	<run_name>-hit<N>/	Workpiece, top die, and bottom die min/max/center coordinates.
load_stroke_hit<N>.csv	<run_name>-hit<N>/	Load-stroke data with columns Step, Time, S, V, Fx, Fy, Fz.

7. Post-Processing One Completed Run

By default, run_deform_hits_config.py calls run_deform_post_processing.py automatically after all hits complete. Set post_process_after_run to false only when you want to skip that automatic step. It is also safe to rerun post-processing manually; existing post-processed CSVs are overwritten with current results.

Spyder procedure:

```
from run_deform_post_processing import process_run

CONFIG_PATH = r"C:\\path\\to\\run_config.json"
process_run(config_path=CONFIG_PATH)
```

Command line procedure:

```
python run_deform_post_processing.py --config C:\\path\\to\\run_config.json
```

If the config does not resolve the completed run folder, pass it directly:

```
python run_deform_post_processing.py --config C:\\path\\to\\run_config.json --run-dir C:\\path\\to\\runs\\my_run_name
```

Post-process a run made with DEFAULT_CONFIG:

```
from run_deform_post_processing import process_run

process_run()
```

```
python run_deform_post_processing.py
```

When run_name is blank in the JSON, the post-processor mirrors the runner's action-file naming rule. If the folder still is not found, use --run-dir or the run_dir argument.

8. Post-Processing Outputs

Output	Meaning
post_processed/initial_microstructure.csv	Initial JMAK grain state. The processor prefers hit 0 because this captures supplier grain initialization before hit 1.
post_processed/microstructure_hit<N>.csv	Element-level GRAIN data parsed from each hit's post/final key.
post_processed/hit_summary.csv	One row per hit with key paths, output paths, grain metrics, load-stroke metrics, and notes.
post_processed/run_metrics.csv	One-row final run summary for comparison, reporting, or later optimization.
post_processed/coordinates_all_hits.csv	Combined coordinate logs for all hits, if present.
post_processed/load_stroke_all_hits.csv	Combined load-stroke CSVs for all hits, if present.
post_processed/damage_summary.csv	Damage min/max from DEFORM API object info or from an existing damage CSV.
post_processed/run_summary.json	Machine-readable record of config path, run path, output files, and processing notes.

9. Die Motion Rules

- If the action filename contains bdm and apply_action_file_rules is true, the runner uses symmetric die motion.

- Symmetric die motion moves the top and bottom dies toward and away from the workpiece together. This is the preferred anti-bending setup.

- If the action filename does not contain bdm, the runner uses single_top_die motion: the top die performs the forming stroke and the bottom die retracts near the end.

- For symmetric motion, the bottom die is positioned before hit 0. For single-top-die motion, the bottom die is normally left at the starting-key position before hit 0.

10. Quality Checks

- After hit 0, inspect the workpiece location relative to both dies before committing to a long run.
- After hit 1, check bending, die contact, and load-stroke output before running many hits.
- Confirm load_stroke_hit<N>.csv has columns Step, Time, S, V, Fx, Fy, Fz.
- Confirm post_processed/hit_summary.csv has one row for every requested forming hit.
- If microstructure columns are missing, confirm the key file contains a GRAIN block followed by ELMDEN.

11. Troubleshooting

Symptom	Likely Cause	Action
Initial key not found	key_path, key_files, die, or use_jmak does not match files on disk	Check the resolved key path and whether the _jmak.key variant exists.
Action CSV missing columns	The config['columns'] names do not match the CSV headers	Update either the config column map or the action CSV headers.
Workpiece bends after one hit	Single-die movement or off-center die/workpiece positioning	Use a bdm action file for symmetric die motion and inspect coordinate logs after hit 0.
No damage_summary.csv	DEFORM API damage extraction unavailable and no existing damage CSV found	Run post-processing on the lab computer with DEFORM API access or export damage to CSV separately.
No microstructure CSV	KEY file lacks JMAK GRAIN data or parsing failed	Use a JMAK key, keep use_jmak true, and inspect the processing notes.

12. Recommended Handoff Package

- The JSON config used for the run.
- The action CSV used for the run.
- The completed run folder containing all hit folders.
- The post_processed folder created by run_deform_post_processing.py.
- A short note describing any visual anomalies observed during DEFORM inspection.